

NAME: Bhavya PS

REG NO: 192524017

COURSE CODE: CSA1716

COURSE NAME: Artificial Intelligence

Artificial Intelligence – Assignment 2

Title: *Solving the 8-Puzzle Problem Using the A Search Algorithm**

1. Brief Introduction to the Problem

The 8-Puzzle Problem is a classic Artificial Intelligence search problem consisting of eight numbered tiles and one blank space arranged on a 3×3 board. The objective is to reach a desired goal configuration by moving one tile at a time into the blank space.

Since many possible board arrangements exist, AI search algorithms are used to determine the optimal sequence of moves.

2. AI Algorithm Suitable for Solving the Problem

A* (A-Star) Search Algorithm

A* is an informed search algorithm that always selects the node with the lowest evaluation function.

Evaluation Function

$$f(n) = g(n) + h(n)$$

Where:

- **$g(n)$** = Cost from the initial state to the current state
- **$h(n)$** = Estimated cost from the current state to the goal state (Heuristic)
- **$f(n)$** = Total estimated cost

Common heuristics:

- Misplaced Tiles
- Manhattan Distance (most commonly used)

3. Step-by-Step Working of the Algorithm

Step 1

Start with the initial state.

2 8 3

1 6 4

7 _ 5

Step 2

Calculate

- $g(n)$
- $h(n)$
- $f(n)$

Step 3

Generate all possible moves.

Possible moves:

- Up
- Left
- Right

Step 4

Calculate the heuristic value for every new state.

Step 5

Choose the state having the smallest $f(n)$.

Step 6

Repeat the above process until the goal state is reached.

Goal State

1 2 3

8 _ 4

7 6 5

4. Algorithm / Pseudocode

Start

Initialize OPEN list with Initial State

Initialize CLOSED list as empty

While OPEN is not empty

 Select node with minimum $f(n)$

 If node is Goal State

 Stop

 Move node to CLOSED list

 Generate successor states

 Calculate

$g(n)$

$h(n)$

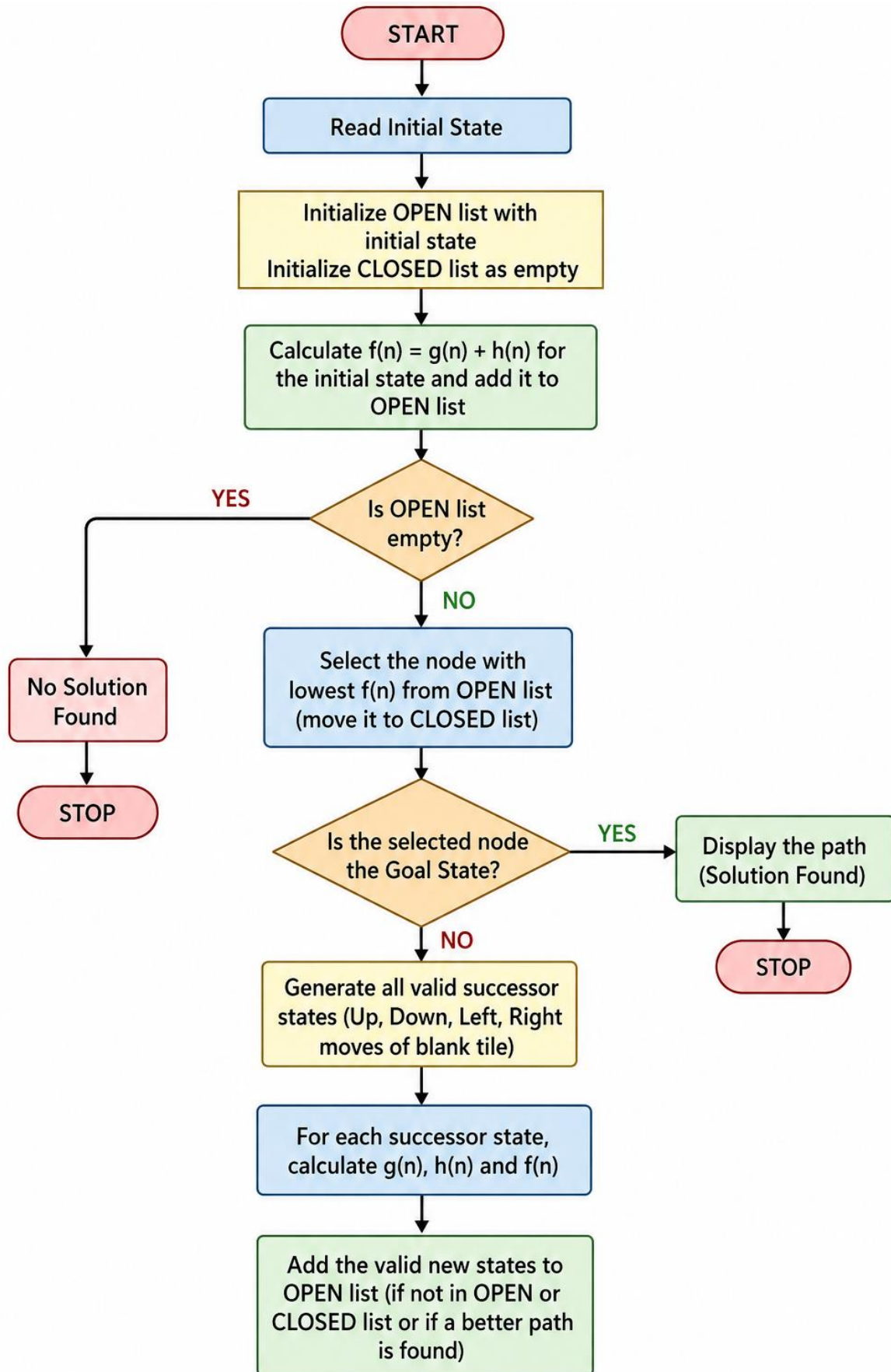
$f(n)$

 Add valid successors to OPEN list

End While

Stop

5. Flowchart



6. Solve One Example Manually

Initial State

2 8 3

1 6 4

7 _ 5

Blank is in the middle of the bottom row.

Possible moves:

Move Left

2 8 3

1 6 4

_ 7 5

Move Right

2 8 3

1 6 4

7 5 _

Move Up

2 8 3

1 _ 4

7 6 5

Among these states, A* computes the heuristic values and selects the state with the smallest $f(n)$. The process continues until the board reaches the goal configuration.

7. Time Complexity

Worst Case:

$O(b^d)$

Where:

- b = Branching factor
- d = Depth of the solution

Although the worst-case complexity is exponential, A* is much more efficient than uninformed search when using a good heuristic.

8. Space Complexity

$O(b^d)$

A* stores all generated nodes in memory, making it memory-intensive.

9. Advantages

- Guarantees the optimal solution when the heuristic is admissible.
- Complete algorithm; it finds a solution if one exists.
- Faster than Breadth-First Search for large search spaces.
- Uses heuristic information to reduce unnecessary exploration.
- Widely used in robotics, navigation systems, and game AI.

10. Limitations

- Requires large memory to store explored nodes.
- Performance depends on the quality of the heuristic function.
- Can be slow for very large state spaces.
- Inefficient when the heuristic provides little guidance.
- Computational cost increases rapidly as the problem size grows.

References

1. Stuart Russell & Peter Norvig, **Artificial Intelligence: A Modern Approach**, 4th Edition, Pearson.
2. Elaine Rich & Kevin Knight, **Artificial Intelligence**, McGraw-Hill.
3. Nils J. Nilsson, **Artificial Intelligence: A New Synthesis**, Morgan Kaufmann.
4. George F. Luger, **Artificial Intelligence: Structures and Strategies for Complex Problem Solving**, Pearson.

This content is suitable for a **4–5 page AI assignment** and covers all the required sections specified in your assignment instructions.